



SPÉCIFICATION TECHNIQUE

Plateforme E-Commerce Pâtisserie

Application Web — Commande en ligne avec personnalisation

Version 1.0 • Mai 2026

Angular

NestJS

PostgreSQL

Supabase

Stripe

Table des matières

1. Présentation du projet

Cette application web permet à une pâtisserie de proposer ses créations en ligne avec un système de commande personnalisé. La plateforme est conçue pour être évolutive : pâtisserie aujourd'hui, bouchées salées et livrets personnalisés demain — tout sur la même plateforme.

1.1 Objectifs

- Interface d'accueil attractive avec appel à l'action fort
- Catalogue produits avec catégories, filtres et détail produit
- Personnalisation de commande (couleur, ingrédients, commentaire) avec répercussion sur le prix
- Espace client : historique et suivi de commandes
- Espace admin : gestion complète des produits, catégories, options et commandes

1.2 Stack technique

Couche	Technologie	Hébergement	Coût
Frontend	Angular 17	Netlify	Gratuit
Backend	NestJS	Railway	Gratuit / ~5\$/mois
Base de données	PostgreSQL	Supabase	Gratuit (500 MB)
Authentification	Supabase Auth	(inclus)	Gratuit
Paieement	Stripe	—	% par transaction
Emails	Resend	—	Gratuit (3 000/mois)

2. Modèle de données — Schéma SQL

Toutes les tables sont dans PostgreSQL (Supabase). Les UUIDs sont générés côté base (`gen_random_uuid()`). Chaque table utilise `updated_at` avec un trigger automatique.

2.1 Table users

Étendue la table `auth.users` de Supabase. Ne pas recréer de table custom — utiliser un trigger pour peupler cette table depuis `auth.users`.

Colonne	Type	Contraintes	Description
<code>id</code>	UUID	PK, FK → <code>auth.users</code>	Identifiant Supabase Auth
<code>email</code>	TEXT	NOT NULL, UNIQUE	Email de connexion
<code>full_name</code>	TEXT	NOT NULL	Nom complet affiché
<code>phone</code>	TEXT	NULLABLE	Numéro de téléphone
<code>role</code>	TEXT DEFAULT 'customer'	CHECK IN ('customer', 'admin')	Rôle applicatif
<code>created_at</code>	TIMESTAMPTZ	DEFAULT now()	Date de création
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT now()	Mise à jour automatique

2.2 Table categories

Colonne	Type	Contraintes	Description
<code>id</code>	UUID	PK	Identifiant unique
<code>name</code>	TEXT	NOT NULL, UNIQUE	Ex : Pâtisserie, Salé, Livrets
<code>slug</code>	TEXT	NOT NULL, UNIQUE	URL-friendly : patisserie, sale
<code>description</code>	TEXT	NULLABLE	Description courte
<code>image_url</code>	TEXT	NULLABLE	Image représentative
<code>sort_order</code>	INTEGER	DEFAULT 0	Ordre d'affichage
<code>is_active</code>	BOOLEAN	DEFAULT true	Visibilité côté client
<code>created_at</code>	TIMESTAMPTZ	DEFAULT now()	
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT now()	

2.3 Table products

Colonne	Type	Contraintes	Description
---------	------	-------------	-------------

id	UUID	PK	Identifiant unique
category_id	UUID	FK → categories.id	Catégorie du produit
name	TEXT	NOT NULL	Nom du produit
slug	TEXT	NOT NULL, UNIQUE	URL-friendly
description	TEXT	NULLABLE	Description longue
base_price	NUMERIC(10,2)	NOT NULL, >= 0	Prix de base en CAD
image_url	TEXT	NULLABLE	Image principale
images	TEXT[]	DEFAULT '{}'	Tableau d'images supplémentaires
is_active	BOOLEAN	DEFAULT true	Activé / désactivé
is_featured	BOOLEAN	DEFAULT false	Mis en avant (accueil)
sort_order	INTEGER	DEFAULT 0	Ordre dans la catégorie
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

2.4 Table product_options

Définit les axes de personnalisation d'un produit (ex: "Couleur", "Parfum", "Taille").

Colonne	Type	Contraintes	Description
id	UUID	PK	Identifiant unique
product_id	UUID	FK → products.id ON DELETE CASCADE	Produit associé
name	TEXT	NOT NULL	Ex : Couleur, Parfum, Taille
type	TEXT	NOT NULL	select text boolean
is_required	BOOLEAN	DEFAULT false	Sélection obligatoire
sort_order	INTEGER	DEFAULT 0	Ordre d'affichage
created_at	TIMESTAMPTZ	DEFAULT now()	

2.5 Table product_option_values

Valeurs possibles pour chaque option de type "select". Le price_modifier est ajouté au prix de base.

Colonne	Type	Contraintes	Description
id	UUID	PK	Identifiant unique

option_id	UUID	FK → product_options.id ON DELETE CASCADE	Option parente
value	TEXT	NOT NULL	Ex : Rouge, Chocolat, 6 personnes
price_modifier	NUMERIC(10,2)	DEFAULT 0	Surcoût ou réduction (+ ou -)
sort_order	INTEGER	DEFAULT 0	Ordre d'affichage

2.6 Table orders

Colonne	Type	Contraintes	Description
id	UUID	PK	Identifiant unique
user_id	UUID	FK → users.id, NULLABLE	NULL si commande invité
status	TEXT	NOT NULL DEFAULT 'pending'	pending confirmed preparing ready delivered cancelled
total_price	NUMERIC(10,2)	NOT NULL	Total calculé au moment de la commande
customer_name	TEXT	NOT NULL	Snapshot du nom (invités inclus)
customer_email	TEXT	NOT NULL	Snapshot email
customer_phone	TEXT	NULLABLE	
delivery_notes	TEXT	NULLABLE	Instructions de livraison / récupération
stripe_payment_intent_id	TEXT	NULLABLE	ID Stripe pour réconciliation
paid_at	TIMESTAMPTZ	NULLABLE	Date de confirmation de paiement
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

2.7 Table order_items

Snapshot complet du produit au moment de la commande — indépendant de toute modification future.

Colonne	Type	Contraintes	Description
id	UUID	PK	
order_id	UUID	FK → orders.id ON DELETE CASCADE	Commande parente

product_id	UUID	FK → products.id, NULLABLE	Référence (peut être NULL si produit supprimé)
product_name	TEXT	NOT NULL	Snapshot du nom
product_image_url	TEXT	NULLABLE	Snapshot image
base_price	NUMERIC(10,2)	NOT NULL	Snapshot prix de base
quantity	INTEGER	NOT NULL, >= 1	Quantité commandée
line_total	NUMERIC(10,2)	NOT NULL	(base_price + options) × quantity
comment	TEXT	NULLABLE	Commentaire libre du client pour cet item

2.8 Table order_item_options

Snapshot des options choisies pour chaque item. Conservé même si les options originales changent.

Colonne	Type	Contraintes	Description
id	UUID	PK	
order_item_id	UUID	FK → order_items.id ON DELETE CASCADE	
option_name	TEXT	NOT NULL	Snapshot nom option (ex : Couleur)
value	TEXT	NOT NULL	Snapshot valeur (ex : Rouge)
price_modifier	NUMERIC(10,2)	NOT NULL DEFAULT 0	Snapshot du surcoût

3. Modèles TypeScript — Backend (NestJS) & Frontend (Angular)

Les interfaces sont partagées via un dossier shared/ ou une librairie commune. Les entités TypeORM côté backend dérivent du même contrat.

3.1 Entités Backend (TypeORM)

User

Propriété	Type	Décorateurs / notes
id	string	@PrimaryGeneratedColumn('uuid')
email	string	@Column({ unique: true })
fullName	string	@Column()
phone	string null	@Column({ nullable: true })
role	'customer' 'admin'	@Column({ default: 'customer' })
orders	Order[]	@OneToMany(() => Order, o => o.user)

Product

Propriété	Type	Décorateurs / notes
id	string	@PrimaryGeneratedColumn('uuid')
category	Category	@ManyToOne(() => Category)
name	string	@Column()
slug	string	@Column({ unique: true })
basePrice	number	@Column('decimal', { precision: 10, scale: 2 })
isActive	boolean	@Column({ default: true })
isFeatured	boolean	@Column({ default: false })
options	ProductOption[]	@OneToMany(() => ProductOption, o => o.product, { eager: true })

3.2 DTOs Backend (NestJS)

Chaque ressource expose 3 DTOs : CreateXxxDto, UpdateXxxDto (Partial<Create>), et une interface de réponse XxxResponseDto.

DTO	Champs principaux	Validation (class-validator)
CreateProductDto	name, description, basePrice, categoryId, imageUrl, isFeatured	@IsString(), @IsNumber(), @IsUUID()
CreateProductOptionDto	productId, name, type, isRequired	@IsEnum(['select', 'text', 'boolean'])
CreateOptionValueDto	optionId, value, priceModifier	@IsNumber(), @Min(-999)

CreateOrderDto	items[], customerName, customerEmail, deliveryNotes	@IsArray(), @ValidateNested()
CreateOrderItemDto	productId, quantity, comment, selectedOptions[]	@IsUUID(), @IsInt(), @Min(1)
UpdateOrderStatusDto	status	@IsEnum(OrderStatus)

3.3 Interfaces Frontend (Angular)

Utilisées dans les services Angular et les composants. Correspondent aux réponses JSON du backend.

Interface	Propriétés
Category	id, name, slug, description, imageUrl, isActive
Product	id, name, slug, basePrice, description, imageUrl, images[], isFeatured, category: Category, options: ProductOption[]
ProductOption	id, name, type, isRequired, values: ProductOptionValue[]
ProductOptionValue	id, value, priceModifier
CartItem	product: Product, quantity: number, selectedOptions: SelectedOption[], comment: string, lineTotal: number
SelectedOption	optionId, optionName, value, priceModifier
Order	id, status, totalPrice, customerName, customerEmail, items: OrderItem[], createdAt, paidAt
OrderItem	id, productName, productImageUrl, basePrice, quantity, lineTotal, comment, options: OrderItemOption[]

4. Implémentations fonctionnelles — Étapes détaillées

4.1 Authentification (Supabase Auth)

Inscription, connexion, récupération de session. Le JWT Supabase est envoyé dans chaque requête au backend NestJS qui le valide.

#	Lieu	Action	Détail
1	Backend	Configurer le guard JWT NestJS	Installer <code>@supabase/supabase-js</code> . Créer <code>JwtStrategy</code> qui appelle <code>supabase.auth.getUser(token)</code> . Injecter dans <code>AuthModule</code> . Décorer les routes protégées avec <code>@UseGuards(JwtAuthGuard)</code> .
2	Backend	Créer le trigger Supabase	Trigger SQL : après INSERT dans <code>auth.users</code> → insérer dans <code>public.users</code> (<code>id</code> , <code>email</code> , <code>full_name</code>). Gérer le cas de conflit (<code>ON CONFLICT DO NOTHING</code>).
3	Frontend	Créer <code>AuthService</code> Angular	Méthodes : <code>signUp(email, password, fullName)</code> , <code>signIn(email, password)</code> , <code>signOut()</code> , <code>getSession()</code> . Utiliser <code>@supabase/supabase-js</code> côté client. Stocker le token en mémoire (<code>BehaviorSubject<Session null></code>).
4	Frontend	Créer <code>AuthGuard</code> Angular	<code>CanActivate</code> : vérifier si session active. Rediriger vers <code>/login</code> sinon. Appliquer sur les routes <code>/dashboard</code> et <code>/admin</code> .
5	Frontend	Pages Login & Register	Formulaires réactifs Angular. Gérer les erreurs Supabase (email déjà utilisé, mot de passe trop court). Redirection post-auth selon le rôle (admin → <code>/admin</code> , client → <code>/</code>).
6	Frontend	Intercepteur HTTP	<code>HttpInterceptor</code> qui récupère le token de la session Supabase et l'ajoute en header <code>Authorization: Bearer <token></code> sur chaque requête vers le backend NestJS.

4.2 Catalogue produits

Affichage des catégories, liste des produits avec filtres, et page détail produit.

#	Lieu	Action	Détail
1	Backend	<code>CategoriesModule</code> : CRUD complet	GET <code>/categories</code> (liste active), GET <code>/categories/:slug</code> , POST <code>/categories</code> (admin), PATCH <code>/categories/:id</code> (admin), DELETE <code>/categories/:id</code> (admin). Service utilise <code>TypeORM Repository<Category></code> .
2	Backend	<code>ProductsModule</code> : liste & filtres	GET <code>/products?category=slug&featured=true&search=mot</code> .

			Utiliser QueryBuilder TypeORM avec WHERE dynamique. Paginer : ?page=1&limit=12. Retourner { data: Product[], total: number, page, limit }.
3	Backend	ProductsModule : détail	GET /products/:slug. Charger le produit avec relations options et option_values (eager loading ou LEFT JOIN). Retourner 404 si slug inexistant ou produit inactif.
4	Frontend	ProductService Angular	getCategories(), getProducts(filters: ProductFilters), getProductBySlug(slug). Utiliser HttpClient. Mettre en cache les catégories avec shareReplay(1).
5	Frontend	Page /products	Afficher les filtres (chips de catégories + barre de recherche). Grille responsive de ProductCardComponent. Pagination. Synchroniser les filtres avec les query params URL (?category=patisserie).
6	Frontend	ProductCardComponent	Inputs : product. Afficher image, nom, prix de base, badge "Nouveau" si isFeatured. Bouton "Voir détails" → navigate vers /products/:slug.
7	Frontend	Page /products/:slug	Charger le produit via le slug depuis l'URL. Afficher galerie images. Rendre les ProductOptionComponent pour chaque option. Calculer le prix dynamiquement. Bouton "Ajouter au panier".

4.3 Personnalisation produit & prix dynamique

Le coeur de l'application. Chaque option peut faire varier le prix affiché en temps réel.

#	Lieu	Action	Détail
1	Frontend	ProductOptionComponent (select)	Input : option: ProductOption. Émet un SelectedOption à chaque changement via @Output(). Afficher les valeurs en radio buttons ou select. Afficher le price_modifier (+2,50 \$ ou -) à côté de chaque choix.
2	Frontend	ProductOptionComponent (text)	Champ texte libre. Émet { optionName, value: texte saisi, priceModifier: 0 }. Placeholder : "Précisez votre choix". Max 200 caractères.
3	Frontend	ProductOptionComponent (boolean)	Checkbox. Si cochée → émet { value: "Oui", priceModifier }. Afficher clairement le surcoût si applicable.
4	Frontend	Champ commentaire libre	Textarea sous les options. Libellé : "Instructions spéciales (allergies, message sur le gâteau...)". Lié à CartItem.comment. Max 500 caractères.
5	Frontend	Calcul du prix dynamique	Dans la page détail produit : prix affiché = product.basePrice +

			sum(selectedOptions.map(o => o.priceModifier)) × quantity. Se recalcule à chaque changement d'option ou de quantité. Animer le changement (transition CSS).
6	Backend	Validation serveur du prix	Dans OrderService.create() : recalculer le prix de chaque item à partir de la base de données (ne jamais faire confiance au prix envoyé par le client). Comparer avec le total reçu. Rejeter si écart > 0,01\$.

4.4 Panier

Le panier est géré entièrement côté frontend (localStorage + service Angular). Aucune table SQL pour le panier.

#	Lieu	Action	Détail
1	Frontend	CartService Angular	BehaviorSubject<CartItem[]>. Méthodes : addItem(product, qty, selectedOptions, comment), updateQuantity(itemIndex, qty), removeItem(itemIndex), clearCart(), getTotal(): number. Persister dans localStorage.
2	Frontend	CartIconComponent (header)	S'abonner au CartService. Afficher le nombre d'items en badge. Cliquer → ouvrir CartDrawer (sidebar overlay).
3	Frontend	CartDrawerComponent	Liste des CartItem avec image, nom, options choisies, commentaire, quantité éditée, bouton supprimer. Total en bas. Bouton "Passer la commande" → /checkout.
4	Frontend	Page /checkout	Formulaire : nom, email, téléphone, notes de livraison. Récapitulatif des items. Total final. Bouton "Payer" → appeler StripeService.

4.5 Paiement Stripe

#	Lieu	Action	Détail
1	Backend	Installer Stripe SDK	npm install stripe. Créer StripeModule avec StripeService. Injecter la clé secrète via ConfigService (variable d'environnement STRIPE_SECRET_KEY).
2	Backend	POST /orders/create-intent	Reçoit CreateOrderDto. Valide et calcule le total. Crée l'Order en base avec status=pending. Crée un PaymentIntent Stripe (amount en centimes, currency: 'cad'). Retourne { orderId, clientSecret }.
3	Frontend	Intégrer Stripe.js	Charger stripe.js via CDN dans index.html. Créer StripeService Angular qui wrap

			loadStripe(publishableKey). Afficher le composant CardElement dans le formulaire de checkout.
4	Frontend	Confirmer le paiement	stripe.confirmCardPayment(clientSecret, { payment_method: { card: cardElement } }). Si succès → naviguer vers /order-confirmation/:orderId. Si erreur → afficher le message d'erreur Stripe.
5	Backend	Webhook Stripe : payment_intent.succeeded	POST /stripe/webhook. Vérifier la signature (stripe.webhooks.constructEvent). Sur succès : mettre à jour orders.status = confirmed, orders.paid_at = now(). Envoyer email de confirmation via Resend.

4.6 Espace client

Accessible uniquement aux utilisateurs connectés. Route protégée par AuthGuard.

#	Lieu	Action	Détail
1	Backend	GET /users/me/orders	Route protégée (JwtAuthGuard). Retourner toutes les commandes de l'utilisateur connecté (extrait du JWT), triées par created_at DESC. Inclure les order_items et order_item_options.
2	Backend	GET /users/me/orders/:id	Vérifier que l'ordre appartient bien à l'utilisateur (user_id = JWT subject). Retourner 403 sinon. Retourner le détail complet.
3	Backend	PATCH /users/me/profile	Permettre la mise à jour de full_name et phone. Valider avec UpdateProfileDto. Ne pas permettre de changer email ou role ici.
4	Frontend	Page /dashboard	Tableau des commandes : date, statut (badge coloré), total, bouton "Voir détail". Statuts avec couleurs : pending=gris, confirmed=bleu, preparing=orange, ready=vert, delivered=vert foncé, cancelled=rouge.
5	Frontend	Page /dashboard/orders/:id	Récapitulatif complet de la commande. Chaque item avec ses options et commentaire. Timeline du statut. Bouton retour.
6	Frontend	Page /dashboard/profile	Formulaire de mise à jour du profil. Pré-rempli avec les données actuelles. Confirmation visuelle après sauvegarde.

4.7 Espace admin

Accessible uniquement au rôle admin. Vérification côté backend (RolesGuard) ET côté frontend (AdminGuard).

#	Lieu	Action	Détail
1	Backend	RolesGuard NestJS	Décorateur <code>@Roles('admin')</code> . Guard vérifie que <code>user.role === 'admin'</code> (extrait du JWT via le profil Supabase). Appliquer sur toutes les routes admin.
2	Backend	CRUD Catégories (admin)	POST <code>/admin/categories</code> , PATCH <code>/admin/categories/:id</code> , DELETE <code>/admin/categories/:id</code> . La suppression vérifie qu'aucun produit actif n'est lié (sinon 400).
3	Backend	CRUD Produits (admin)	POST <code>/admin/products</code> , PATCH <code>/admin/products/:id</code> , DELETE <code>/admin/products/:id</code> (soft delete : <code>is_active=false</code>). Upload images : POST <code>/admin/products/:id/images</code> → stocker dans Supabase Storage.
4	Backend	CRUD Options & Valeurs (admin)	POST <code>/admin/products/:productId/options</code> , PATCH <code>/admin/options/:id</code> , DELETE <code>/admin/options/:id</code> . POST <code>/admin/options/:optionId/values</code> , PATCH <code>/admin/values/:id</code> , DELETE <code>/admin/values/:id</code> .
5	Backend	Gestion commandes (admin)	GET <code>/admin/orders?status=pending</code> (filtrable). PATCH <code>/admin/orders/:id/status</code> (<code>UpdateOrderStatusDto</code>). Envoyer email au client à chaque changement de statut.
6	Frontend	Layout admin	Sidebar avec navigation : Tableau de bord, Catégories, Produits, Commandes. Breadcrumb. Séparé du layout public (<code>AdminLayoutComponent</code>).
7	Frontend	Admin — Tableau de bord	KPIs : commandes du jour, revenus du mois, produits actifs, commandes en attente. Graphique simple des commandes des 30 derniers jours.
8	Frontend	Admin — Gestion produits	Tableau avec tri et recherche. Formulaire produit : champs de base + upload image + section dynamique pour ajouter/modifier/supprimer les options et leurs valeurs. Prévisualisation du prix final.
9	Frontend	Admin — Gestion commandes	Liste filtrée par statut. Cliquer sur une commande → vue détail avec bouton de changement de statut. Afficher les commentaires clients bien visibles.

4.8 Page d'accueil

#	Lieu	Action	Détail
1	Frontend	HeroComponent	Grande image de fond (plein écran). Slogan

			centré. Deux boutons CTA : "Commander maintenant" → /products, "Découvrir nos créations" → scroll vers la section produits. Utiliser @angular/animations pour l'entrée.
2	Frontend	Section produits vedettes	GET /products?featured=true&limit=6. Grille 3 colonnes (desktop) → 2 (tablette) → 1 (mobile). Titre "Nos créations phares".
3	Frontend	Section catégories	GET /categories. Cards cliquables avec image de catégorie. Lien vers /products?category=slug.
4	Frontend	Section "Comment ça marche"	Composant statique. 3 étapes illustrées : 1. Choisissez, 2. Personnalisez, 3. Commandez. Icônes SVG custom.

5. Architecture Backend — Structure NestJS

Module	Responsabilité
AuthModule	JwtStrategy, JwtAuthGuard, RolesGuard. Pas de contrôleur propre — Auth gérée par Supabase.
UsersModule	CRUD profil utilisateur. GET /users/me, PATCH /users/me/profile.
CategoriesModule	CRUD catégories. Routes publiques en lecture, admin en écriture.
ProductsModule	CRUD produits + options + valeurs. Upload images (Supabase Storage). Routes publiques + admin.
OrdersModule	Création commande, calcul prix serveur, gestion statut. Intégration Stripe.
StripeModule	Création PaymentIntent, gestion webhook, service Stripe encapsulé.
EmailModule	Envoi emails via Resend. Templates : confirmation, changement de statut.
AdminModule	Routes admin consolidées. Tableau de bord (stats). Accès restreint RolesGuard.

6. Architecture Frontend — Structure Angular

6.1 Structure des routes

Route	Guard	Composant / Page
/	—	HomePageComponent
/products	—	ProductsPageComponent
/products/:slug	—	ProductDetailPageComponent
/checkout	—	CheckoutPageComponent
/order-confirmation/:id	—	OrderConfirmationPageComponent
/login	—	LoginPageComponent
/register	—	RegisterPageComponent
/dashboard	AuthGuard	DashboardLayoutComponent
/dashboard/orders	AuthGuard	OrdersListPageComponent
/dashboard/orders/:id	AuthGuard	OrderDetailPageComponent
/dashboard/profile	AuthGuard	ProfilePageComponent
/admin	AdminGuard	AdminLayoutComponent
/admin/products	AdminGuard	AdminProductsPageComponent
/admin/categories	AdminGuard	AdminCategoriesPageComponent
/admin/orders	AdminGuard	AdminOrdersPageComponent

6.2 Services Angular

Service	Responsabilité
AuthService	signUp, signIn, signOut, session\$ (BehaviorSubject), currentUser\$
ProductService	getCategories, getProducts(filters), getProductBySlug
CartService	cart\$ (BehaviorSubject), addItem, removeItem, updateQty, clearCart, total\$. Persistance localStorage.
OrderService	createOrder (→ backend), getMyOrders, getOrderById
StripeService	loadStripe, createCardElement, confirmPayment
AdminService	CRUD produits, catégories, options, valeurs, commandes

7. Plan de développement — 3 sprints

Sprint 1 — Fondations & MVP (semaines 1–2)

- Créer le projet NestJS + Angular
- Configurer Supabase (DB + Auth + Storage)
- Écrire et appliquer le schéma SQL complet
- Implémenter CategoriesModule + ProductsModule (lecture)
- Implémenter AuthModule (JWT Guard)
- Frontend : navigation, page accueil, liste produits, détail produit
- Frontend : AuthService + pages login/register
- Déployer sur Netlify (frontend) et Railway (backend)

Sprint 2 — Commandes & Personnalisation (semaines 3–4)

- Frontend : composants d'options et calcul de prix dynamique
- Frontend : CartService + CartDrawer + page Checkout
- Backend : OrdersModule + calcul serveur + validation prix
- Intégration Stripe : PaymentIntent + CardElement + webhook
- Page confirmation de commande
- Espace client : dashboard + liste commandes + détail

Sprint 3 — Admin & Finitions (semaine 5)

- Backend : AdminModule + RolesGuard
- Frontend : layout admin + CRUD produits/catégories/options
- Frontend : gestion commandes admin (statuts)
- EmailModule : Resend (confirmation + changement de statut)
- Tests manuels complets
- Optimisation mobile et accessibilité

8. Variables d'environnement

8.1 Backend (.env)

Variable	Description
DATABASE_URL	URL de connexion PostgreSQL Supabase
SUPABASE_URL	URL du projet Supabase
SUPABASE_SERVICE_ROLE_KEY	Clé service Supabase (backend seulement, jamais exposée)
STRIPE_SECRET_KEY	Clé secrète Stripe (sk_live_... ou sk_test_...)
STRIPE_WEBHOOK_SECRET	Secret de signature pour les webhooks Stripe
RESEND_API_KEY	Clé API Resend pour l'envoi d'emails
FRONTEND_URL	URL du frontend (CORS)

8.2 Frontend (environment.ts)

Variable	Description
apiUrl	URL de base du backend NestJS
supabaseUrl	URL du projet Supabase
supabaseAnonKey	Clé anonyme Supabase (publique)
stripePublishableKey	Clé publique Stripe (pk_...)